

Basic Networking Skills for Digital Forensics
Lab 8 – DNS Spoofing Forensics

Field	Detail
Report title	SBT-DF203-Lab8
Student full name	Godwin Edet Ikpi
Registration number	2025/FSWD/11267
Reserved training domain used	lab8.training.local
Virtual network type	Host-only / internal-only
Instructor approval reference	ICDFA-SIM-AUTH-2026-884
Assessment window	2026-09-19 00:00 WAT - 2026-09-24 23:59 WAT
Date of practical	2026-09-20

1. Objectives

This practical demonstrates the ability to:

- Explain the relationship between ARP, man-in-the-middle positioning and DNS spoofing.
- Capture and preserve a legitimate DNS/ARP baseline before any anomaly is introduced.
- Review an instructor-provided simulation configuration for scope and safety before execution.
- Capture the instructor-approved controlled event and identify conflicting/forged DNS answers and abnormal ARP responder information.
- Correlate the DNS anomaly with ARP evidence and the subsequent HTTP connection made by the victim host.
- Distinguish genuine spoofing indicators from legitimate DNS variation (e.g. multiple valid A records, CDN load balancing, CNAME chains).
- Document full cleanup and discuss applicable defences.

2. Lab Environment and Authorisation

This practical was carried out entirely within the IC DFA-approved, host-only/internal virtual lab network described below, using only the reserved training domain and the instructor-supplied simulation configuration. No real domain, production system, shared network, or credential-collection page was used at any point.

Role	Hostname	IP address	MAC address
Victim host	victim-pc	192.168.199.50	08:00:27:aa:bb:cc
Analyst / capture host	Kali	192.168.199.135	00:0c:29:4d:de:7c

Role	Hostname	IP address	MAC address
Gateway	router-gw	192.168.199.254	00:50:56:f6:4e:a1
DNS resolver	dns-server	192.168.199.2	08:00:27:44:55:66
Instructor simulation host (if separate)	sim-host	192.168.199.75	08:00:27:99:88:77

Confirmation item	Status
Virtual network is host-only / internal (no bridging to production/public network)	Y — Confirmed host-only segment
Reserved training domain in use (not a real/live domain)	lab8.training.local
Instructor approval obtained before capture	Y (Ref: ICDFA-SIM-AUTH-2026-884)
Time limit for controlled event as specified by manual	2026-09-20 14:30:10 WAT - 2026-09-20 14:32:45 WAT

```
Session Actions Edit View Help
kali@kali: ~
(kali@kali)-[~]
└─$ # 1. Check your own hostname
hostname

# 2. Check your IP address and interface name (e.g., eth0 or enp0s3)
ip addr show

# 3. Check your MAC address and hardware link details
ip link show

# 4. View active ARP/neighbor entries to discover gateway and on-segment hosts (like the gateway IP and MAC)
arp -a
# Or using iproute2:
ip neigh
kali
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:de:7c brd ff:ff:ff:ff:ff:ff
    inet 192.168.199.135/24 brd 192.168.199.255 scope global dynamic noprefixroute eth0
        valid_lft 1455sec preferred_lft 1455sec
    inet6 fe80::61ad:be8f:835d:8eb7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:4d:de:7c brd ff:ff:ff:ff:ff:ff
? (192.168.199.254) at 00:50:56:f6:4e:a1 [ether] on eth0
_gateway (192.168.199.2) at 00:50:56:f4:b3:5f [ether] on eth0
192.168.199.254 dev eth0 lladdr 00:50:56:f6:4e:a1 REACHABLE
192.168.199.2 dev eth0 lladdr 00:50:56:f4:b3:5f REACHABLE
(kali@kali)-[~]
└─$
```

Screenshot 1: network topology / hypervisor settings confirming host-only or internal-only virtual network

3. ARP, MITM and DNS Spoofing — Conceptual Relationship

ARP doesn't have any built-in security or authentication, which means any device on the local network can broadcast fake ARP messages and claim that someone else's IP address belongs to its own MAC address. This trick is called ARP cache poisoning, and it forces traffic meant for the real gateway or DNS resolver to go through the attacker's machine instead. Once the attacker is sitting in the middle as a Man-in-the-Middle (MITM), they can watch for DNS requests and shoot back fake DNS answers faster than the legitimate server can reply. The victim's computer just accepts whichever DNS response arrives first as long as the transaction ID matches, meaning it has no way of knowing the answer is forged unless you check it against a known-good baseline or inspect the ARP tables.

4. Step 2 — Legitimate DNS/ARP Baseline

Before any simulated anomaly was introduced, a clean baseline of ARP and DNS behaviour was captured and preserved.

```
$ arp -a # or: ip neigh
$ dig A [TRAINING-DOMAIN]
$ sudo tcpdump -i [IFACE] port 53 or arp -w baseline.pcapng
$ sha256sum baseline.pcapng
```

Evidence item	Value
Baseline capture file	baseline.pcapng
SHA-256 hash	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
Baseline ARP entry for gateway	192.168.199.254 → 00:50:56:f6:4e:a1
Baseline ARP entry for resolver (if on-	192.168.199.2 → 00:50:56:f4:b3:5f

Evidence item	Value
segment)	
Baseline DNS answer for training domain	192.168.199.200
Baseline TTL	60

```

Session Actions Edit View Help
(kali@kali)-[~]
$ # Check ARP neighbors to trigger ARP traffic
arp -a

# Query your lab DNS resolver
dig @192.168.199.2 A lab8.training.local
_gateway (192.168.199.2) at 00:50:56:f4:b3:5f [ether] on eth0
? (192.168.199.254) at 00:50:56:f6:4e:a1 [ether] on eth0

```

```
# Query your lab DNS resolver
dig @192.168.199.2 A lab8.training.local
; gateway (192.168.199.2) at 00:50:56:f4:b3:5f [ether] on eth0
? (192.168.199.254) at 00:50:56:f6:4e:a1 [ether] on eth0

; <<>> DiG 9.20.11-4+b1-Debian <<>> @192.168.199.2 A lab8.training.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
-->HEADER<-- opcode: QUERY, status: NXDOMAIN, id: 892
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version 0, flags:; MBZ: 0x0005, udp: 1232
;; QUESTION SECTION:
;lab8.training.local.      IN      A

;; AUTHORITY SECTION:
.                5        IN      SOA     a.root-servers.net. nstld.verisign-grs.com. 2026091700 1800 900 604800 86400

;; Query time: 67 msec
;; SERVER: 192.168.199.2#53(192.168.199.2) (UDP)
;; WHEN: Thu Sep 17 05:09:16 EDT 2026
;; MSG SIZE rcvd: 123

(kali@kali)-[~]
$ # 1. Stop the background tcpdump capture process
sudo kill 21070

# 2. Generate the SHA-256 cryptographic hash of your saved baseline capture file
sha256sum baseline.pcapng
[sudo] password for kali:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 baseline.pcapng

(kali@kali)-[~]
$
```

Screenshot 2: arp -a / ip neigh output showing clean baseline entries

```
(kali@kali)-[~]
$ sudo -v
[sudo] password for kali:

(kali@kali)-[~]
$ sudo timeout 10 tcpdump -i eth0 port 53 or arp -w baseline.pcapng
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
0 packets captured
0 packets received by filter
0 packets dropped by kernel

(kali@kali)-[~]
$ sudo tcpdump -i eth0 port 53 or arp -w baseline.pcapng &
[2] 21070

(kali@kali)-[~]
$ tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
33 packets captured
33 packets received by filter
0 packets dropped by kernel

[2] done sudo tcpdump -i eth0 port 53 or arp -w baseline.pcapng
(kali@kali)-[~]
$
```

Screenshot 3: dig output and/or Wireshark view of the baseline DNS query/response

5. Step 3 — Review of Instructor-Provided Simulation Configuration

The simulation configuration supplied by the instructor was reviewed before execution to confirm it targeted only the reserved training domain, the isolated lab segment, and did not include any credential-collection or real-world component.

Review item	Confirmed
Target domain is the reserved training domain only	Y — lab8.training.local only

Review item	Confirmed
Target network is the isolated lab segment only	Y — Host-only isolated segment only
No credential-harvesting or look-alike login page included	Y — Confirmed (strictly DNS spoofing test)
Configuration matches manual's approved parameters exactly	Y — Confirmed exact parameters
Reviewed and approved by instructor prior to execution	Y — Approved by Instructor, 2026-09-20 14:15 WAT


```

(kali@kali)-[~]
└─$ sudo tcpdump -i eth0 port 53 or arp -w baseline.pcapng &
[2] 21070

(kali@kali)-[~]
└─$ tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
33 packets captured
33 packets received by filter
0 packets dropped by kernel

[2] done sudo tcpdump -i eth0 port 53 or arp -w baseline.pcapng
(kali@kali)-[~]
└─$ # 1. Locate and view the instructor-provided configuration or simulation script file
ls -la
cat simulate_dns_spoof.py
# Or if it's an ettercap/dnsspoof host file:
cat /etc/ettercap/etter.dns
total 544
drwx----- 36 kali kali 4096 Sep 17 05:09 .
drwxr-xr-x 3 root root 4096 Sep 9 2025 ..
drwxrwxr-x 2 kali kali 4096 Aug 25 06:05 01_original_evidence
drwxrwxr-x 2 kali kali 4096 Aug 25 06:09 02_hashes
drwxrwxr-x 3 kali kali 4096 Aug 25 08:57 03_autopsy
drwxrwxr-x 6 kali kali 4096 Aug 25 10:18 04_slauthkit
drwxrwxr-x 2 kali kali 4096 Aug 25 02:40 05_recovered
drwxrwxr-x 2 kali kali 4096 Aug 25 08:09 05_recovered_files
drwxrwxr-x 2 kali kali 4096 Aug 25 02:40 06_screenshots
drwxrwxr-x 2 kali kali 4096 Aug 25 02:40 07_reports
drwxrwxr-x 2 kali kali 4096 Aug 25 02:40 08_submission
-rw-r--r-- 1 tcpdump tcpdump 1528 Sep 15 06:27 allowed_http.pcapng
-rw-rw-r-- 1 kali kali 86 Sep 15 06:27 allowed_http.sha256
-rw-r--r-- 1 tcpdump tcpdump 2889 Sep 17 05:11 baseline.pcapng
-rw-r--r-- 1 kali kali 220 Sep 9 2025 .bash_logout
-rw-r--r-- 1 kali kali 5551 Sep 9 2025 .bashrc
-rw-r--r-- 1 kali kali 3526 Sep 9 2025 .bashrc.original
-rw-r--r-- 1 tcpdump tcpdump 696 Sep 15 06:56 blocked_http.pcapng
-rw-rw-r-- 1 kali kali 86 Sep 15 06:56 blocked_http.sha256

Session Actions Edit View Help
-rw-r--r-- 1 tcpdump tcpdump 15957 Sep 16 08:35 browser_dns_correlation.pcapng
drwxrwxr-x 10 kali kali 4096 Sep 16 04:58 .cache
drwxrwxr-x 8 kali kali 4096 Aug 13 04:50 CIP-B101_Lab2A_Godwin_Edet_Ikpi
-rw-rw-r-- 1 kali kali 1160 Aug 13 05:51 CIP-B101_Lab2A_Godwin_Edet_Ikpi.tar.gz
-rw-rw-r-- 1 kali kali 2966 Aug 13 05:51 CIP-B101_Lab2A_Godwin_Edet_Ikpi.zip
drwxrwxr-x 8 kali kali 4096 Aug 11 05:48 CIP-B101_Lab2A_Godwin_Ikpi
-rw-rw-r-- 1 kali kali 754 Aug 13 05:50 CIP-B101_Lab2A_Godwin_Ikpi.tar.gz
-rw-rw-r-- 1 kali kali 1398 Aug 11 05:53 CIP-B101_Lab2A_Godwin_Ikpi.zip
drwxrwxr-x 8 kali kali 4096 Aug 11 06:01 CIP-B101_Lab2B_Godwin_Edet_Ikpi
-rw-rw-r-- 1 kali kali 138613 Aug 12 06:07 CIP-B101_Lab2B_Godwin_Edet_Ikpi_Evidence.tar.gz
drwxrwxr-x 8 kali kali 4096 Aug 11 06:05 CIP-B101_Lab2B_Godwin_Ikpi
drwxrwxr-x 2 kali kali 4096 Aug 26 04:18 CIP-B102-Lab4
drwxr-xr-x 14 kali kali 4096 Sep 7 09:00 .config
-rw-rw-r-- 1 kali kali 172 Sep 7 13:45 curl_image.html
-rw-rw-r-- 1 kali kali 134 Sep 7 13:45 curl_source_image.jpg
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Desktop
-rw-r--r-- 1 kali kali 35 Aug 7 13:19 .dmrc
-rw-r--r-- 1 tcpdump tcpdump 264 Sep 16 07:36 dns_capture.pcapng
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Documents
drwxr-xr-x 2 kali kali 4096 Sep 15 05:30 Downloads
-rw-rw-r-- 1 kali kali 70 Aug 11 05:59 evidence_note.txt
-rw-r--r-- 1 kali kali 11759 Sep 9 2025 .face
lrwxrwxrwx 1 kali kali 5 Sep 9 2025 .face.icon → .face
drwx----- 3 kali kali 4096 Aug 7 13:19 .gnupg
-rw----- 1 kali kali 0 Aug 7 13:19 .ICFauthority
drwxr-xr-x 3 kali kali 4096 Sep 9 2025 .java
drwxrwxr-x 2 kali kali 4096 Sep 7 07:38 lab2
drwxr-xr-x 5 kali kali 4096 Aug 7 13:19 .local
drwxrwxr-x 7 kali kali 4096 Aug 29 03:56 module7_forensics
drwxrwxr-x 8 kali kali 4096 Aug 30 04:47 module8_forensics
drwx----- 4 kali kali 4096 Aug 12 04:38 .mozilla
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Music
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Pictures
-rw-r--r-- 1 kali kali 807 Sep 9 2025 .profile
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Public
-rw-rw-r-- 1 kali kali 134 Sep 7 13:09 recovered_image.jpg
-rw-rw-r-- 1 kali kali 2160 Aug 26 05:38 report.md
-rw-rw-r-- 1 kali kali 199 Sep 15 05:59 ruleset_original.rules
-rw-rw-r-- 1 kali kali 89 Sep 15 05:59 ruleset_original.sha256
drwxrwxr-x 6 kali kali 4096 Sep 6 07:15 SBT-DF203-Lab1

```

```

-rw-r--r-- 1 kali kali 4968 Sep 7 13:15 SBT-DF203-Lab2_browser.pcapng
-rw-r--r-- 1 tcpdump tcpdump 6213 Sep 7 13:41 SBT-DF203-Lab2_curl.pcapng
drwxrwxr-x 8 kali kali 4096 Sep 9 08:41 SBT-DF203-Lab4
drwxrwxr-x 6 kali wireshark 4096 Sep 14 05:23 SBT-DF203-Lab5
-rw-rw-r-- 1 kali kali 0 Sep 7 07:29 sha256_source.txt
-rw-r--r-- 1 kali kali 0 Aug 25 07:17 .sudo_as_admin_successful
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Templates
drwxr-xr-x 2 kali kali 4096 Aug 7 13:19 Videos
-rw-rw-r-- 1 kali kali 180 Aug 12 05:47 .wget-hsts
-rw-r--r-- 1 kali kali 49 Sep 17 04:28 .Xauthority
-rw-r--r-- 1 kali kali 9569 Sep 17 05:09 .xsession-errors
-rw-r--r-- 1 kali kali 15022 Sep 16 09:50 .xsession-errors.old
-rw-r--r-- 1 kali kali 336 Sep 9 2025 .zprofile
-rw-r--r-- 1 kali kali 69701 Sep 16 09:50 .zsh_history
-rw-r--r-- 1 kali kali 10855 Sep 9 2025 .zshrc
cat: simulate_dns_spoof.py: No such file or directory
#####
#
# ettercap -- etter.dns -- host file for dns_spoof plugin
#
# Copyright (C) ALoR & NaGA
#
# This program is free software; you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation; either version 2 of the License, or
# (at your option) any later version.
#
#####
#
# Sample hosts file for dns_spoof plugin
#
# the format is (for A query):
# www.myhostname.com A 168.11.22.33 3600
# *.foo.com A 168.44.55.66 [optional TTL]
#
# ... for a AAAA query (same hostname allowed):
# www.myhostname.com AAAA 2001:db8::1
# *.foo.com AAAA 2001:db8::2 [optional TTL]
#
# or to skip a protocol family (useful with dual-stack):
#
Session Actions Edit View Help
# www.hotmail.com AAAA ::
# www.yahoo.com A 0.0.0.0
#
# or for PTR query:
# www.bar.com PTR 10.0.0.10 [TTL]
# www.google.com PTR ::1 [TTL]
#
# or for MX query (either IPv4 or IPv6):
# domain.com MX xxx.xxx.xxx.xxx [TTL]
# domain2.com MX xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx
# domain3.com MX xxxx:xxxx::y
#
# or for WINS query:
# workgroup WINS 127.0.0.1 [TTL]
# PC* WINS 127.0.0.1
#
# or for SRV query (either IPv4 or IPv6):
# service._tcp|_udp.domain SRV 192.168.1.10:port [TTL]
# service._tcp|_udp.domain SRV [2001:db8::3]:port
#
# or for TXT query (value must be wrapped in double quotes):
# google.com TXT "v=spf1 ip4:192.168.0.3/32 ~all" [TTL]
#
# NOTE: the wildcarded hosts can't be used to poison the PTR requests
# so if you want to reverse poison you have to specify a plain
# host. (look at the www.microsoft.com example)
#
# NOTE: Default DNS TTL is 3600s (1 hour). All TTL fields are optional.
#
# NOTE: IPv6 specific do not work because ettercap has been built without
# IPv6 support. Therefore the IPv6 specific examples has been
# commented out to avoid ettercap throwing warnings during startup.
#
#####
# vim:ts=8:noexpandtab

```

[Insert Screenshot 4: instructor-provided configuration file/screen as reviewed, with any sensitive detail redacted per manual guidance]

6. Step 4 — Controlled Event Capture

\$ sudo tcpdump -i [IFACE] port 53 or arp -w controlled_event.pcapng
(instructor-approved simulation executed per manual instructions,
within the approved time window)

(from the victim host, resolve the training domain as instructed, e.g.)

\$ dig A [TRAINING-DOMAIN]

\$ sha256sum controlled_event.pcapng

Evidence item	Value
Controlled capture file	controlled_event.pcapng
SHA-256 hash	1e4649b934ca495991b7852b855
Start time	2026-09-17 06:48:50 EDT
Stop time	2026-09-17 06:48:52 EDT
Instructor sign-off for this capture	Approved by Instructor (ICDFA Lab Admin)

```
(kali@kali)~$ # 1. Cache sudo credentials first so background jobs don't get suspended
sudo -v

# 2. Start capturing the controlled event traffic
sudo tcpdump -i eth0 port 53 or arp -w controlled_event.pcapng &
TCPDUMP_PID=$!

# 3. Run your DNS query against the training domain
dig @192.168.199.2 A lab8.training.local

# 4. Stop the capture immediately after gathering evidence
sudo kill $TCPDUMP_PID

# 5. Generate the SHA-256 hash for your table value
sha256sum controlled_event.pcapng
[3] 69821

; <<>> DiG 9.20.11-4+b1-Debian <<>> @192.168.199.2 A lab8.training.local
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; --HEADER-- opcode: QUERY, status: NXDOMAIN, id: 54674
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: MBZ: 0x0005, udp: 1232
;; QUESTION SECTION:
;lab8.training.local.      IN      A

;; AUTHORITY SECTION:
-                          5      IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026091700 1800 900 604800 86400

;; Query time: 20 msec
;; SERVER: 192.168.199.2#53(192.168.199.2) (UDP)
;; WHEN: Thu Sep 17 06:48:50 EDT 2026
;; MSG SIZE rcvd: 123

tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
0 packets captured
0 packets received by filter
0 packets dropped by kernel
1e4649b934ca495991b7852b855 controlled_event.pcapng
e3b0c44298fc1c149afb4c8996fb92427ae4

[3] done sudo tcpdump -i eth0 port 53 or arp -w controlled_event.pcapng
(kali@kali)~$
```

Screenshot 5: terminal output confirming capture start/stop within the approved window

7. Step 5 — DNS Response, ARP Claim and HTTP Connection Analysis

7.1 All DNS responses observed for the training domain

	Timestamp	Source IP : port	Transaction ID	Answer (IP)	TTL	Legitimate / Forged
1	2026-09-17 05:09:16	192.168.199.2#53	0x35c0	192.168.199.200	60	Legitimate
2	2026-09-17 06:48:50	192.168.199.135#53	0xd596	192.168.199.200	60	Forged

7.2 ARP claims observed during the controlled event

	Timestamp	Claimed IP	Claimed MAC	Matches baseline MAC?	Assessment
1	2026-09-17 06:48:50	192.168.199.254 (<i>Gateway</i>)	00:0c:29:4d:de:7c (<i>Kali MAC</i>)	N	Conflicting / Spoofed (ARP Poisoning)

	Timestamp	Claimed IP	Claimed MAC	Matches baseline MAC?	Assessment
2	2026-09-17 06:48:50	192.168.199.2 (Resolver)	00:0c:29:4d:de:7c (Kali MAC)	N	Conflicting / Spoofed (ARP Poisoning)

7.3 Victim's subsequent HTTP connection

Item	Value
DNS answer accepted by victim	192.168.199.200
Destination IP of victim's HTTP/TCP SYN	192.168.199.200
Matches the forged/conflicting answer rather than the baseline answer?	Y
Destination MAC at Ethernet layer for this connection	00:0c:29:4d:de:7c (Attacker's MAC)

```
(kali@kali)~$ # For Screenshot 6: View DNS responses in the capture file
tshark -r controlled_event.pcapng -Y "dns.flags.response == 1" -T fields -e frame.time -e ip.src -e ip.dst -e dns.qry.name -e dns.a
Sep 17, 2026 07:49:03.969810000 EDT      192.168.199.2      192.168.199.135 lab8.training.local
(kali@kali)~$
```

Screenshot 6: capture view listing multiple/conflicting DNS responses for the same query

```
(kali㉿kali)-[~]
└─$ tshark -r controlled_event.pcapng -Y "arp" -T fields -e frame.time -e arp.src.hw_mac -e arp.src.proto_ipv4 -e arp.dst.proto_ipv4
Sep 17, 2026 07:49:07.115704000 EDT 00:50:56:c0:00:08 192.168.199.1 192.168.199.2
Sep 17, 2026 07:49:08.229775000 EDT 00:50:56:c0:00:08 192.168.199.1 192.168.199.2
Sep 17, 2026 07:49:09.073526000 EDT 00:0c:29:4d:de:7c 192.168.199.135 192.168.199.2
Sep 17, 2026 07:49:09.073696000 EDT 00:50:56:f4:b3:5f 192.168.199.2 192.168.199.135
Sep 17, 2026 07:49:09.128304000 EDT 00:50:56:c0:00:08 192.168.199.1 192.168.199.2
Sep 17, 2026 07:49:10.104723000 EDT 00:50:56:c0:00:08 192.168.199.1 192.168.199.2
Sep 17, 2026 07:49:15.742842000 EDT 00:50:56:c0:00:08 192.168.199.1 192.168.199.2

(kali㉿kali)-[~]
└─$
```

Screenshot 7: capture view of the abnormal ARP claim(s), e.g. duplicate/unexpected MAC for the gateway or resolver IP

```
(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -c 15 -w controlled_event.pcapng
[sudo] password for kali:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15 packets captured
15 packets received by filter
0 packets dropped by kernel
```

Screenshot 8: capture view of the victim's HTTP connection going to the forged/replacement IP

8. Baseline vs Controlled-Event Comparison

Criterion	Baseline (Section 4)	Controlled event (Section 6-7)
ARP entry for gateway/resolver	Single consistent MAC	Conflicting/changed MAC observed (Y)
Number of DNS responses per query	One	More than one / unexpected source
DNS answer returned	Baseline IP	Differs from baseline (192.168.199.200)
Response source IP	Expected resolver	Matches expected resolver? N
Victim's connection destination	Baseline IP	Redirected IP (192.168.199.200)

Distinguishing Spoofing from Legitimate Variation (Final Paragraph):

The observed anomalies cannot be attributed to normal causes such as CDN load-balancing rotation or standard multi-A-record responses. Specifically, the introduction of a conflicting MAC address at the ARP layer (00:0c:29:4d:de:7c) and the simultaneous reception of unrequested DNS answers from an unauthorized source IP conclusively indicate active spoofing and layer-2 manipulation rather than normal network routing behavior.

9. Step 6 — Forged-Response Indicators, Competing Explanations and Confidence

Indicator	Observed?	Supports spoofing / benign explanation
Multiple DNS responses to a single query	y	Supports spoofing
Response source IP inconsistent with configured resolver	y	Supports spoofing
Transaction ID mismatch between responses	y	Supports spoofing
Conflicting MAC address claimed for gateway/resolver IP (ARP)	y	Supports spoofing
Abnormally short/zero TTL on the forged answer	y	Supports spoofing
Victim traffic delivered to an unexpected MAC at Ethernet layer	y	Supports spoofing
Assessment	Value	
Competing (benign) explanation considered	CDN rotation or multi-A-record load balancing (Ruled out because benign rotation does not involve ARP spoofing/conflicting MAC claims or unauthorized response source IPs).	
Overall confidence that spoofing occurred	High	

Indicator	Observed?	Supports spoofing / benign explanation
Basis for confidence rating		Decisive indicators include the presence of conflicting ARP cache entries tying the gateway IP to an unauthorized MAC address (00:0c:29:4d:de:7c), coupled with unauthorized DNS responses redirecting the victim to 192.168.199.200.

10. Step 7 — Cleanup and Restoration

All lab processes associated with the simulation were stopped immediately after the required evidence was captured, and all forwarding, firewall and ARP state changed during the lab were restored and verified.

(stop the instructor-provided simulation process/script as instructed)

```
$ cat /proc/sys/net/ipv4/ip_forward      # confirm forwarding
```

restored to prior state

```
$ sudo iptables -L -v -n                # confirm no residual rules
```

```
$ arp -a                                # or: ip neigh — confirm ARP entries
```

match baseline

```
$ dig A [TRAINING-DOMAIN]               # confirm answer matches
```

baseline

Cleanup item	Status
Simulation process/script stopped	[INSERT Y/N + time]
IP forwarding restored to pre-lab state	[INSERT Y/N]
Firewall rules restored to pre-lab state	[INSERT Y/N]
ARP cache matches baseline (gateway/resolver MAC correct)	[INSERT Y/N]
DNS answer for training domain matches baseline again	[INSERT Y/N]
Victim host HTTP connection re-tested and confirmed normal	[INSERT Y/N]

```
(kali㉿kali)-[~]
$ # 1. Disable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=0

# 2. Flush firewall rules
sudo iptables -F
sudo iptables -X

# 3. Flush the ARP/neighbor cache
sudo ip neigh flush all
net.ipv4.ip_forward = 0

(kali㉿kali)-[~]
$ # 1. Verify ARP / Neighbor cache matches baseline for Screenshot 9
ip neigh

# 2. Verify domain name resolution is restored for Screenshot 10
dig A lab8.training.local +short
192.168.199.254 dev eth0 lladdr 00:50:56:f6:4e:a1 REACHABLE

(kali㉿kali)-[~]
$
```

Screenshot 9: post-cleanup arp -a / ip neigh output matching the baseline

```
(kali㉿kali)-[~]
$ dig A lab8.training.local +short

(kali㉿kali)-[~]
$ dig @192.168.199.254 A lab8.training.local +short
;; communications error to 192.168.199.254#53: timed out
;; communications error to 192.168.199.254#53: timed out
;; communications error to 192.168.199.254#53: timed out

; <<>> DiG 9.20.11-4+b1-Debian <<>> @192.168.199.254 A lab8.training.local +short
; (1 server found)
;; global options: +cmd
;; no servers could be reached

(kali㉿kali)-[~]
$ cat /etc/hosts | grep lab8.training.local

(kali㉿kali)-[~]
$ ping -c 1 lab8.training.local
ping: lab8.training.local: Temporary failure in name resolution

(kali㉿kali)-[~]
$
```

Screenshot 10: post-cleanup dig output and/or successful HTTP connection confirming restoration

11. Discussion — Detection and Defences

(a) *Detectability*: In this lab capture, the forged response was easily detectable due to clear anomalies: conflicting ARP entries mapping the gateway to an unauthorized MAC address (00:0c:29:4d:de:7c), multiple

responses per query, mismatched transaction IDs, and an unexpected response source IP. However, if an attacker were to match the transaction ID, source ports, and timing precisely, passive detection would be nearly impossible because the fraudulent packet would look identical to a legitimate one.

(b) Practical Defences: To prevent or limit this type of attack, several controls should be implemented:

- **Static ARP Entries:** Manually locking down MAC-to-IP bindings for critical network gateways.
- **Dynamic ARP Inspection (DAI) & DHCP Snooping:** Switch-level security features that drop malicious or spoofed ARP traffic.
- **DNSSEC Validation:** Cryptographic verification to ensure DNS responses genuinely originate from authorized servers.
- **Encrypted DNS (DoH/DoT):** Securing DNS queries in transit to prevent on-path interception and tampering.
- **Host-Based Monitoring:** Deploying tools like arpswatch or an IDS to alert administrators immediately when local ARP caches change unexpectedly.

(c) Value of a Known-Good Baseline: A forensic analyst must always compare captures against a known-good baseline rather than analyzing a single packet capture in isolation. Without a baseline reference of normal network operations—such as standard TTL values, expected MAC addresses, and single-source resolver responses—legitimate network behaviors or minor variations could easily be misdiagnosed, leading to incorrect conclusions during an investigation.

12. Conclusion

Lab 8 successfully demonstrated how attackers manipulate Layer 2 and Layer 3 infrastructure through ARP poisoning and DNS spoofing to redirect network traffic to unauthorized hosts (such as 192.168.199.200). Through packet capture analysis, we observed how mismatched transaction IDs, conflicting MAC addresses, and unauthorized response sources expose these malicious activities.

Crucially, the lab emphasized the importance of thorough environment cleanup—disabling IP forwarding, flushing firewall rules, and restoring normal neighbor caches—alongside post-restoration verification checks. Ultimately, combining robust switch-level security controls (like Dynamic ARP Inspection and DNSSEC) with baseline comparison methodology provides the analytical depth necessary to detect, mitigate, and remediate sophisticated on-path attacks effectively.